StayOps India Product Requirements Document (PRD)

Document Status: Draft for Development Studio Review
Project Code Name: StayOps India
Product Type: B2B SaaS for Indian homestays, villas, and short-term vacation rentals
Primary Market: India
Prepared For: Product, Engineering, UI/UX, QA, and Delivery Teams

---

1. Executive Summary

StayOps India is a mobile-first property operations and booking platform designed for the Indian homestay, villa, and short-term rental segment. The product is positioned around three structural pain points: remote ownership, semi-skilled ground staff, and fragmented inventory across OTAs and direct channels.

The product combines an owner-facing PMS, a caretaker operations app, a two-way channel manager, and a direct-booking stack spanning WhatsApp-first commerce and a lightweight web storefront. The commercial objective is to reduce OTA commission leakage, improve occupancy control, and create a recurring subscription SaaS business with predictable monthly revenue.

This PRD structures the concept into a development-ready document with clear scope boundaries, implementation priorities, non-functional requirements, operational assumptions, pricing logic, delivery milestones, and acceptance-oriented requirements. It is intended to reduce ambiguity before technical estimation, solution architecture, sprint planning, and contract scoping.

---

2. Product Vision

StayOps India aims to become the operating system for independent hospitality operators who do not fit the standard hotel PMS model. The concept specifically targets owners who often live in metro cities while operating properties in remote leisure destinations, where on-ground execution depends on caretakers and local staff rather than trained hotel teams.

The product vision is therefore not only “booking management,” but “remote property control with direct revenue capture.” The software must help an owner manage availability, bookings, housekeeping readiness, petty cash, meal billing, and guest communication from a single operational platform.

---

3. Problem Statement

Three structural operational issues define the target market:

- **Remote Management:** Owners are remote from the property, which creates weak real-time visibility into operations, readiness, and cash handling.
- **Staff Usability:** Local staff need low-text, mobile-first workflows and often require regional language support.
- **Inventory Fragmentation:** OTA and direct booking inventory fragmentation creates a high risk of double bookings and slow response times.

In practical terms, these issues create revenue leakage, guest experience inconsistency, and owner distrust of ground operations. Any development effort should treat calendar integrity, ease of use, and low-friction booking conversion as the core product pillars.

---

4. Users and Roles

4.1 Primary User Types

- **Property Owner:** Monitors occupancy, pricing, cash spends, guest movement, and channel performance.
- **Property Manager:** Manages one or more properties, staff, and booking operations across a portfolio.
- **Caretaker / Ground Staff:** Executes housekeeping, guest check-in/check-out, meal logging, and expense capture.
- **Guest:** Interacts mainly through WhatsApp flows or the web booking interface.
- **Admin / Support Team:** Oversees onboarding, subscriptions, audit visibility, and exception handling.

4.2 Role Design Implication

Role-based access control must be designed from the beginning. Permissions should separate owner and caretaker workflows cleanly and should not be treated as a post-MVP enhancement.

---

5. Product Scope

5.1 In Scope for MVP

- Core PMS with property, room, booking, calendar, and guest records.
- Caretaker mobile workflows for housekeeping readiness, check-in/check-out, meal logging, and expense capture.
- Lite website/storefront generated automatically from PMS content.
- Direct web booking widget with payment gateway integration.
- WhatsApp booking-assist flow for inquiry capture and booking progression.
- Two-way inventory synchronization using API where available and iCal fallback where required.
- Owner dashboards for revenue, occupancy, and operational updates.
- Subscription billing engine and plan enforcement.

5.2 Out of Scope for MVP

- Advanced revenue management with AI-driven dynamic pricing.
- Full CRM and marketing automation beyond core WhatsApp notification and booking conversation flows.
- Accounting-grade ERP functionality and GST reconciliation.
- Native iOS-first optimization beyond cross-platform framework support.
- Marketplace integrations beyond the core listed OTA set.

---

6. Functional Requirements

6.1 Direct Booking System

The direct-booking experience is split into two functional streams sharing a single source of truth for inventory, pricing, and booking state.

WhatsApp Engine Requirements

- Guest initiates inquiry through click-to-chat links, QR codes, or property profile entry points.
- Bot captures check-in date, check-out date, guest count, and optionally room/property preference.
- System returns live availability with pricing and property details based on PMS inventory.
- Payment request link is generated through gateway integration and tied to a booking intent.
- Successful payment confirmation auto-creates the booking, issues a voucher, and updates all calendars.

Lite Web Storefront Requirements

- Website content is statically generated or cached from PMS-managed property data.
- Booking widget must query live availability and pricing before checkout.
- Tier-based support for subdomain and custom domain mapping must be enforced.
- Mobile performance must remain a core acceptance criterion to support browsing under weak network conditions.

6.2 Core PMS and Caretaker App

The caretaker product prioritizes low-text density, large tap targets, visual statuses, and simple task flows to match the target user profile.

Mandatory Operational Capabilities

- Property and room inventory management.
- Availability calendar with booking source tagging.
- Check-in/check-out status actions with real-time owner notifications.
- Housekeeping checklist completion with required photo evidence before “Ready” status can be updated.
- Expense capture with bill image upload and petty cash deduction logic.
- Meal order logging against an active guest stay or booking.
- Language toggle support for regional ground staff.

6.3 Channel Manager

The channel manager serves as a critical control layer. Booking timestamp integrity, sync retries, audit logs, and conflict handling need explicit engineering treatment.

Minimum Requirements

- Support synchronization across Airbnb, Booking.com, Agoda, and MakeMyTrip.
- Use direct API/XML where channel access is available and iCal polling fallback where needed.
- Maintain conflict resolution logic using booking timestamp priority and immediate calendar block propagation.
- Log every inbound and outbound sync event for troubleshooting and support operations.

---

7. Non-Functional Requirements

7.1 Security

- Multi-tenant data isolation is mandatory at the database level.
- All PII and payment-adjacent data must be encrypted in transit and at rest.
- Role-based access control must govern owner, manager, caretaker, and admin actions.
- Audit logs must track booking changes, financial entries, and permission-sensitive actions.
- No raw card storage; payments must rely entirely on compliant external gateways.

7.2 Performance

- Calendar availability reads should feel near real-time to users.
- OTA sync jobs must be retry-safe and idempotent.
- Booking creation must use row-level locks to prevent duplicate inventory allocation under concurrent demand.
- Lite website pages must remain optimized for low-bandwidth mobile browsing.

7.3 Reliability

- Booking and payment webhook handling must be resilient to duplicate events (idempotency tokens).
- Sync failures should trigger visible operational alerts on the admin/owner dashboard.
- Media upload workflows should support delayed background sync when caretaker connectivity is weak.

7.4 Localization and Usability

- Android-first usability should guide early mobile UI/UX decisions.
- Language switching should be designed at the content model level, not hard-coded into screens.
- Caretaker actions should minimize free-text entry wherever possible in favor of visual selection.

---

8. Pricing and Packaging

8.1 Recommended Pricing Structure

Plan	Monthly Price	Annual Price	Intended User	Property Limit	Key Inclusions
Lite	INR 1,499	INR 14,990	Single apartment / small homestay	1 property, up to 3 rooms	Core PMS, subdomain website, standard WhatsApp alerts, iCal sync, basic revenue dashboard
Pro Growth	INR 2,999	INR 29,990	Standalone villa / premium homestay	1 property, unlimited rooms	Full PMS, custom domain, booking bot, real-time API sync where supported, expense and food ledger, advanced profit dashboard
Multi-Property	INR 5,999	INR 59,990	Property manager /	Up to 5 properties	Everything in Pro plus portfolio roll-up dashboard, centralized

Plan	Monthly Price	Annual Price	Intended User	Property Limit	Key Inclusions
Hub			operator		oversight, multi-property controls

8.2 Pricing Improvements Recommended for Development

- **Annual Billing Discount:** Provides approximately two months free equivalent, improving upfront cash collection.
- **Overage Rules:** Additional property pricing should be defined as a flat custom add-on fee for the Multi-Property Hub beyond the 5-property cap.
- **Setup Fees:** Define a one-time assisted onboarding fee for custom domain setup, channel onboarding, and catalog import under Pro and Multi-Property plans.
- **WhatsApp Costs:** Conversation or utility-message usage costs should be governed by a fair-use policy or treated as a pass-through cost if usage exceeds thresholds.
- **Payment Gateways:** Gateway transaction fees must be explicitly excluded from SaaS subscription fees.

8.3 Commercial Guardrails

- Define whether “property” and “room” are distinct billable entities in the database architecture.
- Implement hard vs soft limits for room caps and property caps within the subscription middleware.
- Clarify trial logic, downgrade behavior, grace periods, and failed payment handlings before writing the billing module code.

---

9. System Architecture

9.1 Architecture Principles

- Single source of truth for inventory and booking state.
- Event-driven handling of booking, payment, and channel sync actions.
- Strict tenant isolation in the data model.
- API-first backend so mobile, web, and integrations reuse the same domain logic.
- Background job processing for sync, notifications, website generation, and document delivery.

9.2 Core Services

- Identity and Access Service (RBAC)
- Property and Inventory Service
- Booking and Availability Engine
- Payment and Invoicing Integration Layer
- Channel Manager Sync Service
- WhatsApp Messaging Workflow Service
- Media and Document Service
- Reporting and Analytics Service

---

10. Data Model Guidance

The development studio should formalize the schema with the following mandatory attributes:

- Room/unit types and dynamic inventory mapping.
- Booking source metadata and sync event history references.
- Task checklist templates customizable by property or room type.
- Expense categories and approval states.
- Subscription plan, add-on, invoice, and payment status records.
- Notification logs for WhatsApp, push, and email events.
- Immutable booking event history for auditability.

---

11. Integrations

11.1 Mandatory Integrations

- **Payment Gateways:** Razorpay or Cashfree integration.
- **OTA/Channel Connectivity:** Direct API or iCal mappings for Airbnb, Booking.com, Agoda, and MakeMyTrip.
- **Messaging:** WhatsApp Cloud API for conversational booking flows and utility notifications.
- **Media Storage:** S3-compatible cloud storage for vouchers, bill photos, and housekeeping evidence.

11.2 Integration Risks

- OTA API access may vary by partner eligibility and commercial developer agreements.
- iCal-based sync is inherently delayed compared to direct APIs and must be treated as a best-effort near-real-time fallback.
- WhatsApp template approval rules and Meta billing updates could alter unit economics if unmonitored.

---

12. Assumptions and Dependencies

12.1 Assumptions

- Target customers are willing to operate booking workflows entirely on mobile devices.
- Caretakers possess Android devices with intermittent but usable data connectivity.
- Customers accept a subscription-first SaaS model over commission-led transaction structures.
- Properties already maintain the basic digital assets (photos, descriptions, rates) needed for storefront generation.

12.2 Dependencies

- Third-party approval and API credentials for channel managers and Meta WhatsApp business accounts.
- Stable payment webhook delivery from the chosen payment provider.
- UI/UX design system optimized specifically for low-literacy operational roles.

---

13. Risks and Mitigations

Risk	Impact	Mitigation
Calendar sync delays create double bookings	High	Maintain strict booking lock logic, retry queues, conflict alerts, and clear operational logging.
OTA API limitations slow rollout	High	Ship phased support with iCal fallback and explicit channel compatibility matrices.
Caretaker adoption is weak due to UI complexity	High	Run field-tested Android-first UX with regional language support and photo-driven workflows.
WhatsApp cost or policy changes affect margins	Medium	Introduce fair-use policy and plan-based message thresholds.
Pricing is under-scoped for support-heavy customers	Medium	Introduce setup fees, premium onboarding, and an add-on structure early.
Multi-tenant security gaps create trust risk	High	Enforce strict RBAC, audit logs, encryption, and secure tenant boundaries from day one.

14. Delivery Roadmap

Phase 1: Core Platform (Weeks 1-6)

Deliver tenant model, core booking engine, calendar state machine, payment integration foundations, and owner dashboard skeleton.

Phase 2: Distribution and Storefront (Weeks 7-12)

Deliver OTA connectivity, sync monitoring queues, auto-generated storefront, and live booking widget.

Phase 3: Caretaker and WhatsApp Workflows (Weeks 13-18)

Deliver caretaker application UI, localized interface capability, WhatsApp inquiry-to-booking flow, and operational notifications.

Phase 4: Beta, QA, and Hardening (Weeks 19-24)

Deliver pilot onboarding tools, production environment hardening, monitoring/observability stacks, bug fixes, and launch readiness.

---

15. Acceptance Criteria Guidance

Sprint-ready user stories must define and satisfy the following:

- Booking creation under high concurrent requests prevents overbooking.
- Availability sync failures trigger fallback queues and admin notifications immediately.
- Required-photo enforcement prevents a room status change to "Ready" if empty.
- Expense entries require an image attachment before deducting from petty cash records.
- Website regeneration finishes within target execution limits after a content update.
- Plan entitlement enforcement blocks actions if room or property limits are breached.

16. Open Questions for Development Studio

1. Should WhatsApp support full end-to-end booking in MVP, or only inquiry qualification followed by payment link issuance?
2. Which OTA integrations are technically and commercially viable for phase one vs later phases?
3. Will custom domain onboarding be self-serve, assisted, or fully managed?
4. What is the target SLA for sync intervals on iCal-based channels?
5. Should the product support owner-only web access initially, or both owner and manager personas from launch?
6. Will invoices, vouchers, and guest communications require multi-language templates at launch?
7. What support model is assumed in the pricing tiers: email only, chat, or assisted onboarding?